

Why Computer Science?

Christoph Kirsch

University of Salzburg, Austria

Czech Technical University in Prague, Czechia

Everything that follows is built from one distinction: the difference between a **proof** and the **truth**.

Those two colours keep those two jobs for the whole half hour: **proof**, **notation**, **syntax** — **truth**, **meaning**, **semantics**.

FULL DISCLOSURE

This entire talk was generated by an AI.

It took me 40+ years to become someone who could prompt it into existence. I could do that today. I could not have done it yesterday.

That sentence is the whole argument. The next half hour is its **derivation**.

"Why study computer science when a machine will **write the code** for me?"

THE CHEAP ANSWER

Jobs, salaries, the next product cycle. All of it true today, all of it a forecast — and forecasts expire on the next product cycle. You deserve better than that.

THE REAL ANSWER · STATED NOW, EARNED LATER

Using a tool that generates **notation** requires knowing what the notation **means** and whether it is **true**. That knowledge has a name: a discipline. Computer science is the discipline whose *subject* is languages and their meaning.

THE CLAIM

*Only knowing and understanding a nontrivial domain — its languages and their semantics — enables efficient and effective use of generative AI, and of whatever follows it. Modern AI does not make computer science optional. It motivates studying it **more** than before.*

The rest of the talk proves this from results that were true before this technology arrived and will still be true after whatever replaces it: Cantor, Gödel, Turing, Rice, Cook. And it shows you the system we teach it with.

Six parts.

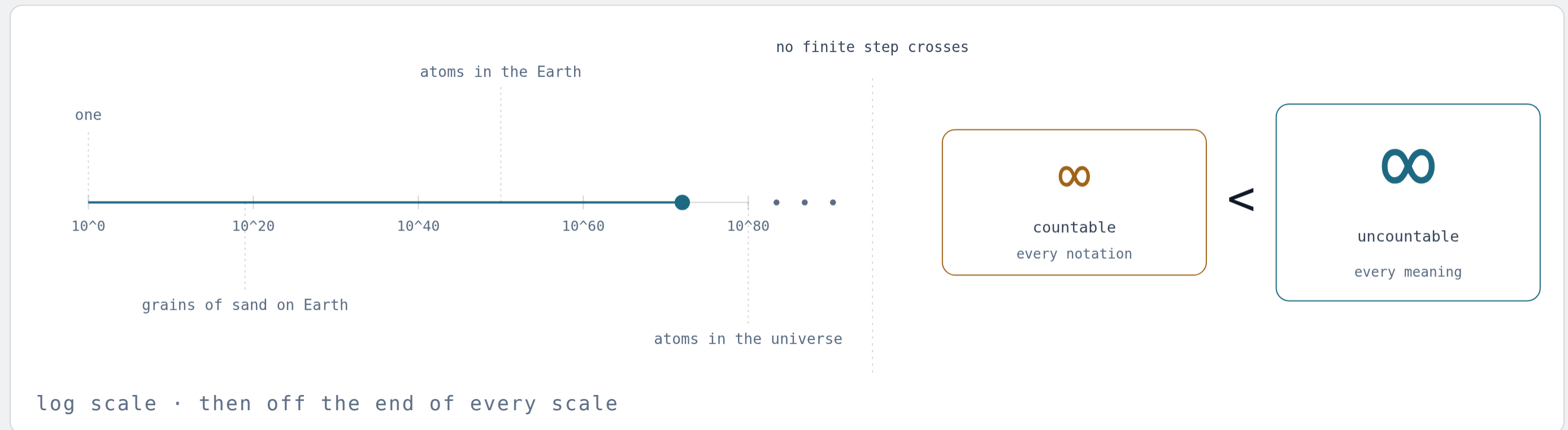
I	Size	why 34 bytes beat the universe
II	Infinity	why meanings outnumber notations
III	Self-reference	why proof $\neq$ truth · and where the computer came from
IV	Cost	hard to find, easy to check
V	Machines	what today's AI cannot escape
VI	Why computer science	the claim, earned · and where to start

Four of these parts end in a theorem that sounds like bad news. Every one of them turns out to be a **job description**. That reversal is the point of the lecture.

Size

First a feeling for how big the spaces are that programs live in — because everything later, including every bug and every AI, happens inside one.

One line carries the whole talk: small, vast, then two sizes of endless.



Left. From one thing to every atom there is — the whole physical world in eighty steps, each one ten times the last.

The wall. No number of steps crosses it. Infinity is not the far end of the line; it is what the line never reaches.

Right. Past the wall, endlessness comes in sizes: one counts every notation, the other every meaning — and is **strictly bigger**.

A bit is **one distinction**. Add one and you **double** the states — so **34 bytes** beat the universe.

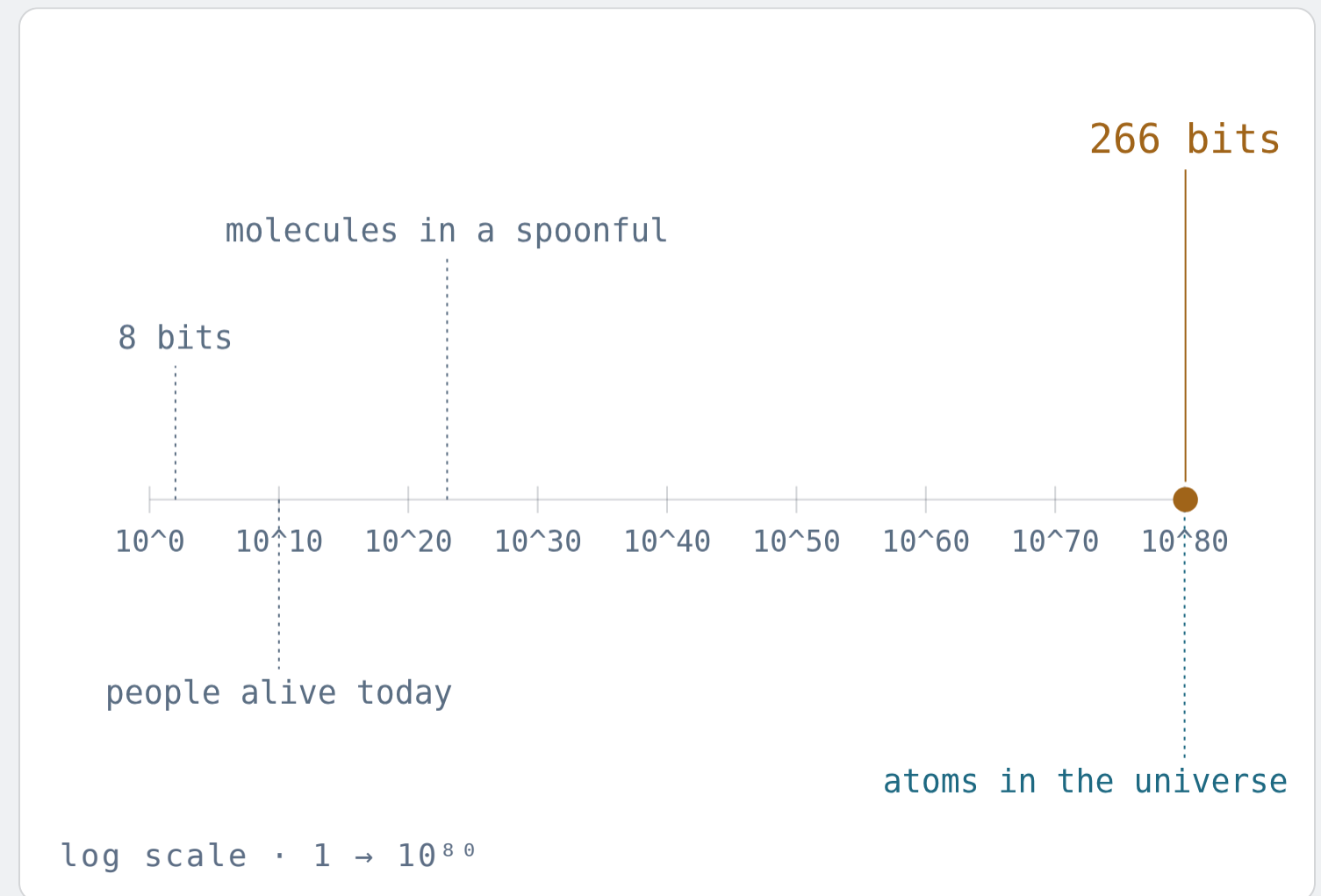
Ten bits: a thousand states. Twenty: a million.
Thirty: a billion. Doubling is the most underestimated operation in human reasoning — and the entire engine of computing.

10^{80}

ATOMS IN THE OBSERVABLE UNIVERSE

2^{266}

STATES OF 266 BITS $\approx 1.2 \times 10^{80}$



Selfie is 12,394 lines. Its machine has $2^{34,359,738,368}$ states.

Selfie is the system this course is taught with: one file of C, containing a compiler, an emulator and a hypervisor, each of which is applied to *itself*. You can read all of it. In one semester, you will have.

The machine it runs on has 4 GB of memory — 2^{35} bits. Written out in decimal, its state count has about 10.3 billion digits: 3.4 million pages, some 6,900 volumes of 500 pages. That shelf does not hold the states. It holds *the number* of them, written down once.

Every program selfie will ever run, every bug you will ever write for it, is one point in that space.

```
$ ./selfie -c selfie.c
selfie compiling selfie.c to 64-bit RISC-U with 64-bit starc
365784 characters read in 12394 lines and 1741 comments
491 global variables, 661 procedures, 512 string literals
188392 bytes generated with 43492 instructions
```

```
2^34359738368 = 238731173716868705040517371867534379031689797475
3805346507569675911986411509205033986994105429253718329002209917
5144573637279641592487667345504934445665896266534390508060257364
5887873587205645363686153927830933717047214016855749948381371876
3620556826923243128918037860241764100458175521435595493998354279
8793703970552420972448293831600998200108824043991385179964603310
0713066205001254701178001213707758667360474660002581816302872523
```

In a space that big, good states and bad states look alike.



Test a billion states per second, starting at the Big Bang, and by now you would have checked about 2^{88} of them — out of $2^{34,359,738,368}$.

The fraction is not small. It is **indistinguishable from zero**.

Testing shows the presence, not the absence of bugs.

EDSGER W. DIJKSTRA, 1969

So building software cannot be exhaustive search. It has to be argument, structure, abstraction — which is to say, **notation**. That is the seed of Part III.

Infinity

Vast is still finite. Now the things that are not — and the discovery that endlessness comes in **two sizes**.

NOTATION

Everything you can write down is countable.

Two sets are the same size if you can pair them up, one to one, nothing left over. A set pairable with 1, 2, 3, ... is countable: it can be listed.

A program is a finite string of symbols. So is a proof, a specification, a prompt. List all strings of length 1, then 2, then 3 ... every finite text appears at some finite position.

So all programs that will ever exist form one list. Same for all proofs. Same for all prompts anyone will ever type.

1.	ϵ
2.	0
3.	1
4.	00
5.	01
6.	10
7.	11
8.	000
9.	001
10.	...

every finite text
has a number

the enumeration of all texts

THIS HAS A NAME

Giving each piece of text a number is *Gödelisierung* — Gödel numbering. `selfie.c` is 365,784 characters: one string, one very long number. It is how a machine reads a machine, and Part III is built on it.

THE PROOF

Hand me a list of every behaviour.
I will build one that is **not on it**.

	1	2	3	4	5	6
S1	○	●	●	●	○	●
S2	●	●	○	○	○	○
S3	○	○	●	●	○	○
S4	○	○	○	○	●	○
S5	●	●	●	○	○	●
S6	○	○	●	●	●	○
D	●	○	○	●	●	●

D disagrees with row n about n

● yes · ○ no

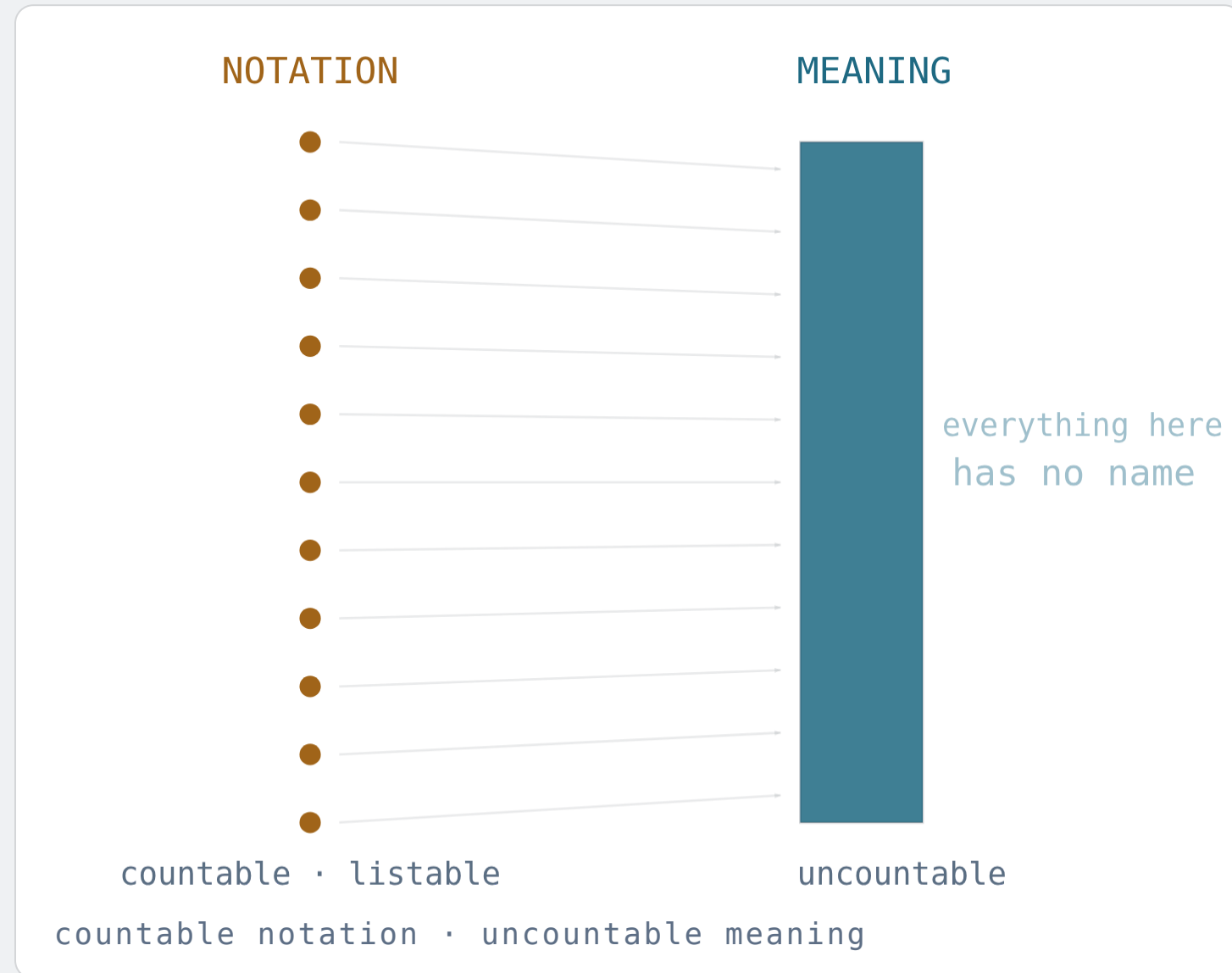
A behaviour is an endless answer sheet: for input 1, yes or no; for input 2, yes or no; forever. Every infinite bit string is one. Suppose they could be listed: S1, S2, S3, ..., every behaviour somewhere on the list.

Go down the **diagonal** and ask each row about *its own number*. What does S1 say about 1? S2 about 2? Sn about n?

Build D by answering the opposite every time: $D(n) = \text{not } S_n(n)$. Nothing exotic — a rule anyone can apply, one number at a time.

D is not S1: they disagree about 1. Not S2: they disagree about 2. Not Sn, for any n. So the list left something out — and it was any list at all. Cantor, 1891.

There are incomparably more meanings than notations.



Programs: countable. Behaviours: uncountable. So almost every behaviour has no program.

Proofs: countable. **Truths** about numbers: uncountable. So almost every truth has no proof.

Prompts: countable. Things you might mean by one: uncountable. So almost everything is unsaid.

READ AS A JOB DESCRIPTION

No vocabulary — mathematical, legal, neural — will ever cover the space of meanings. Every new notation captures meaning that was unreachable before, and there is always more left. The supply never runs out.

Self-Reference

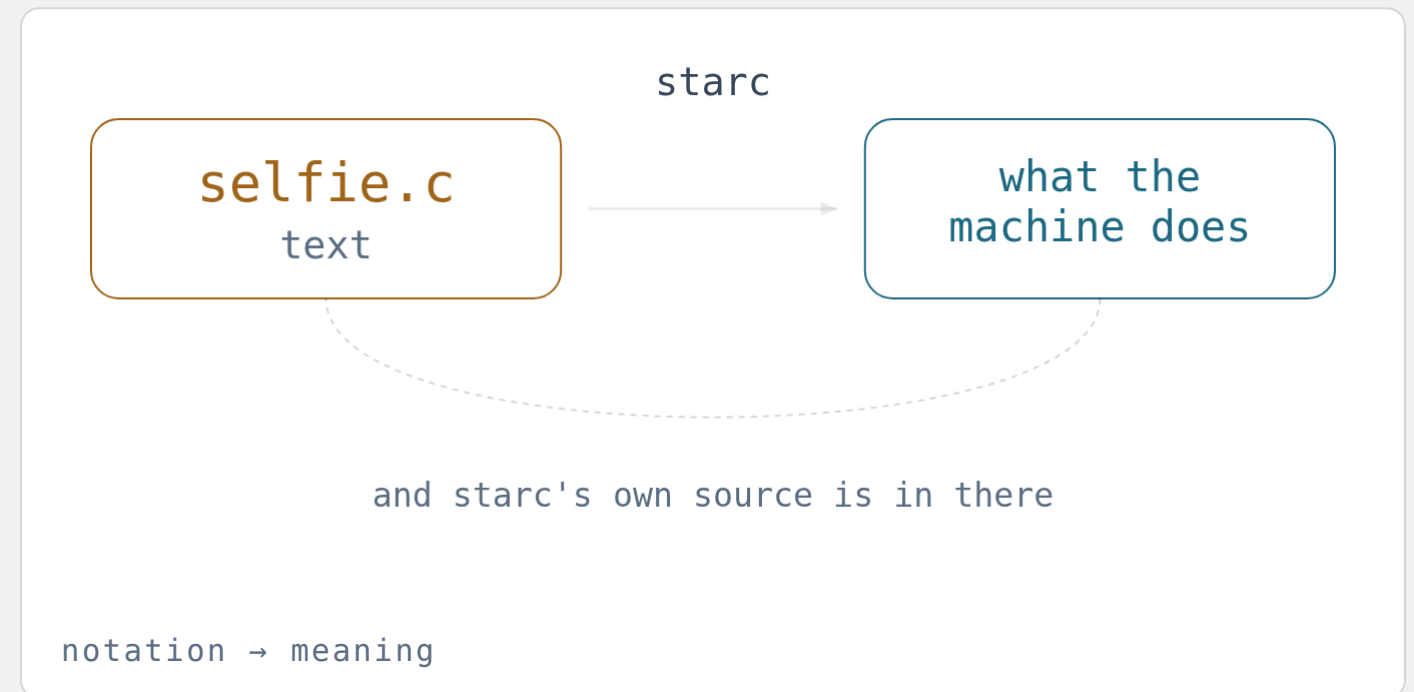
Counting says the gap exists. Self-reference walks to the edge and [points](#) — and the same step, taken forwards, builds the computer.

A compiler defines the meaning of the language it is **written in**.

What does `while` mean? Not what the manual says. What it means is whatever the compiler *emits* — a comparison and a branch — and whatever the machine then does with those.

So the meaning of a program is fixed by another program. And selfie's compiler, `starc`, is written in the very language it gives meaning to.

An English dictionary written in English. Only paradoxical if you demand that the definitions come first. They do not: **the machine comes first**, and the dictionary is bootstrapped onto it.



THE DISTINCTION, IN THE ROOM'S OWN TERMS

Syntax is the text — finite, checkable, copyable. **Semantics** is what the machine does with it — over all inputs, forever. A semantics is a *function* from the first to the second, and pinning that function down is the founding act of every exact discipline.

Three commands. Three kinds of self-reference.

```
// self-compilation: the compiler compiles its own source – twice – and the results agree
$ ./selfie -c selfie.c -o selfie1.m -m 2 -c selfie.c -o selfie2.m
$ diff -s selfie1.m selfie2.m
Files selfie1.m and selfie2.m are identical
```

```
// self-execution: the emulator executes its own machine code
$ ./selfie -c selfie.c -o selfie.m -m 2 -l selfie.m -m 1
```

```
// self-hosting: the hypervisor hosts a virtual machine running the hypervisor
$ ./selfie -c selfie.c -o selfie.m -m 3 -l selfie.m -y 2 -l selfie.m -y 1
```

The first is **Gödel's** move: a text handed to itself. The second is **Turing's**: a machine handed its own description. The third is what an **operating system** is. All three run on a laptop, tonight.

THEOREM

There are **truths** with **no proof** — and no strong system can certify **itself**.

GÖDEL · INCOMPLETENESS · 1931

*Any consistent formal system rich enough to describe arithmetic contains statements that are **true** but not **provable** within it. And it cannot prove its own consistency.*

HOW · THE SAME DIAGONAL

By Gödelisierung, build a sentence that says "*this sentence has no proof in this system.*" If it were provable, the system proves a falsehood. So it is **unprovable** — and therefore **true**. Adding it as an axiom does not help: the construction runs again.

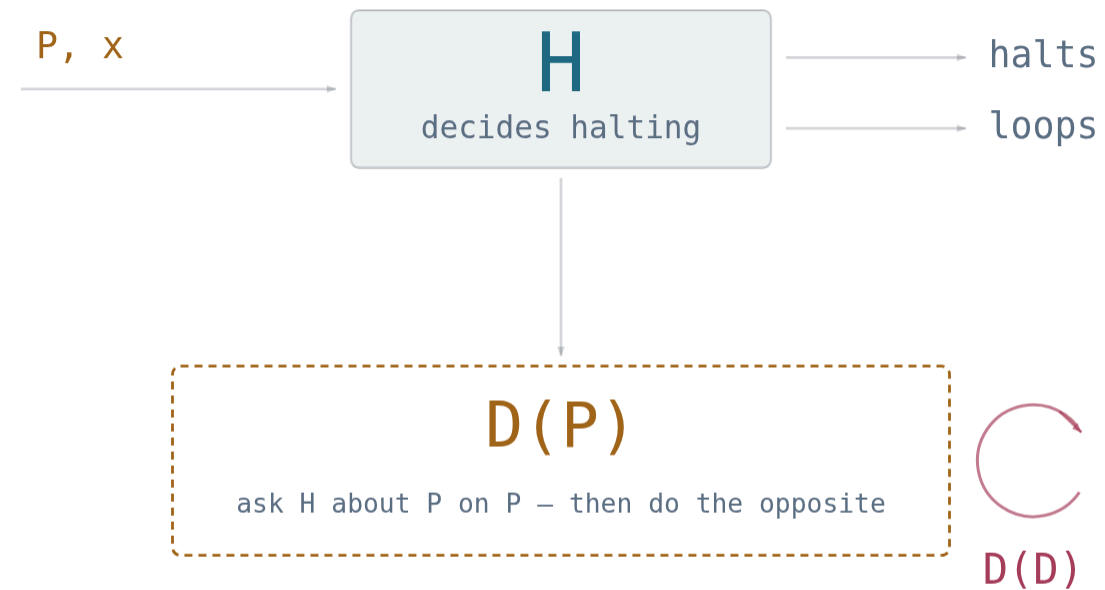
READ IT AS ENGINEERING · THOMPSON 1984

Selfie's fixed point proves the compiler agrees with *itself*. It cannot prove the binary does what the source says: a compiler can carry a back door that reinserts itself on every self-compilation, invisible in clean source. The check has to come from outside — a second, independent compiler.

Proof is a finite object you can check. **Truth** is not. **Proof is syntax**. **Truth is semantics**. They are not the same size — and nothing certifies itself.

UNDECIDABILITY

Will this program ever **stop**?
No program can always tell.



$\text{halts} \iff \neg \text{halts}$ — so H cannot exist

feed the decider its own description

Suppose $H(P, x)$ decides, for every program and input, whether P halts on x .

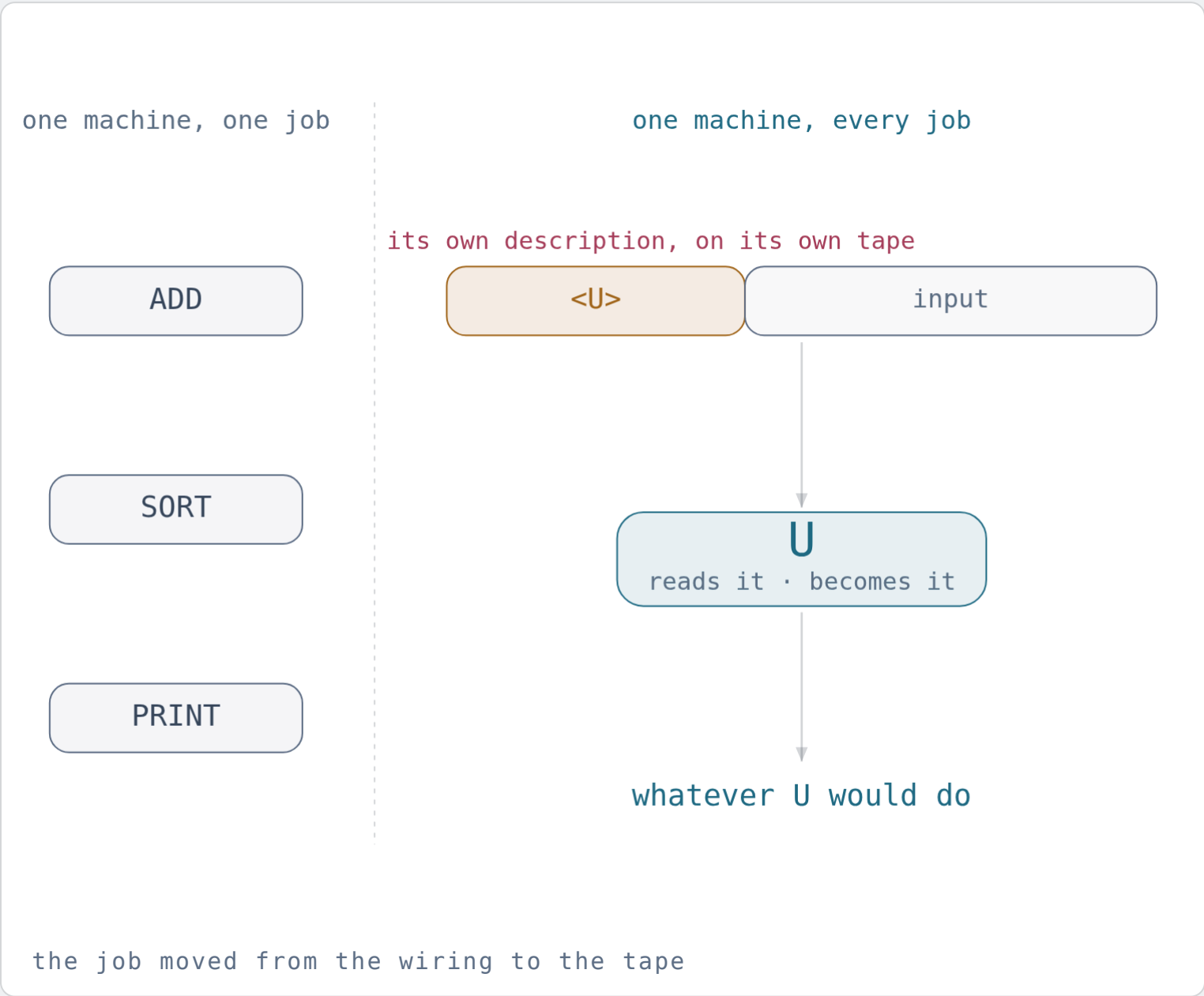
Build $D(P)$: ask H whether P halts on P — then do the opposite.

Now run $D(D)$. It halts exactly if it doesn't.

Contradiction. So H never existed.

Turing, 1936 — the same paper that defined the universal machine, and thus invented the computer. The limit and the machine arrived together.

One machine that can be any machine.



A machine used to *be* its job. Turing's move: put the machine's description *on the tape*, as data. Then one machine U reads any description and does whatever that machine would do. That is universality: one piece of hardware, every possible behaviour, because the behaviour arrives as **notation**.

Your phone is not phone-shaped. It is U holding a description — which is why a new app needs no new hardware.

SELFIE'S U IS A PAGE OF C PER INSTRUCTION GROUP
mipster reads a word of RISC-U code, does what it says, and advances. Fourteen instructions, and it runs every program that will ever exist for that machine — including itself. That is the second of the three commands.

ONE SENTENCE, READ TWICE
Hand a decider its own description: contradiction, no H. Hand a machine *any* description: every program at once, one U. Turing published both in 1936.

THE GENERAL CASE

It is not just halting. It is **every interesting question about meaning.**

RICE · 1953

*Every non-trivial property of the behaviour of programs is undecidable.
Only questions about their text are safe.*

DECIDABLE · ABOUT SYNTAX

Does it parse? How long is it? Which procedures does it call?
Are the types consistent? Does it contain a loop? A compiler
answers all of these, always, in finite time.

UNDECIDABLE · ABOUT SEMANTICS

Does it terminate? Is it equivalent to that one? Can it divide by
zero? Does it leak the key? Is it correct? No program decides
any of these for all programs — whoever, or whatever, wrote
them.

So every useful tool — a type checker, a test suite, a fuzzer, a model checker, a proof assistant — is a deliberate approximation: sound but incomplete, or complete but unsound, or exact only within a bound. Choosing which to give up is the discipline. Selfie ships several such choices, applied to selfie.

PATTERN

One idea. Sixty years. Six theorems.

YEAR	WHO	THE LIST	THE DIAGONAL OBJECT
1891	Cantor	all behaviours	a row on no list
1901	Russell	all sets	the set of non-self-members
1931	Gödel	all provable sentences	"I am unprovable"
1936	Tarski	all definable predicates	"I am false"
1936	Turing	all decidable questions	a program that defies its judge
1953	Rice	all semantic properties	all of the above, at once

Assume a complete list. Ask each item about *itself*. Answer the opposite.
Learn the move once and you own the century.

Cost

Suppose a question *is* decidable. You still have to pay for the answer — in time, space, and energy.

DECIDABLE $\neq$ DOABLE

A question with a guaranteed answer you will **never receive**.

Given a logical formula over 100 yes/no variables: is there an assignment making it true? Perfectly decidable — try all 2^{100} .

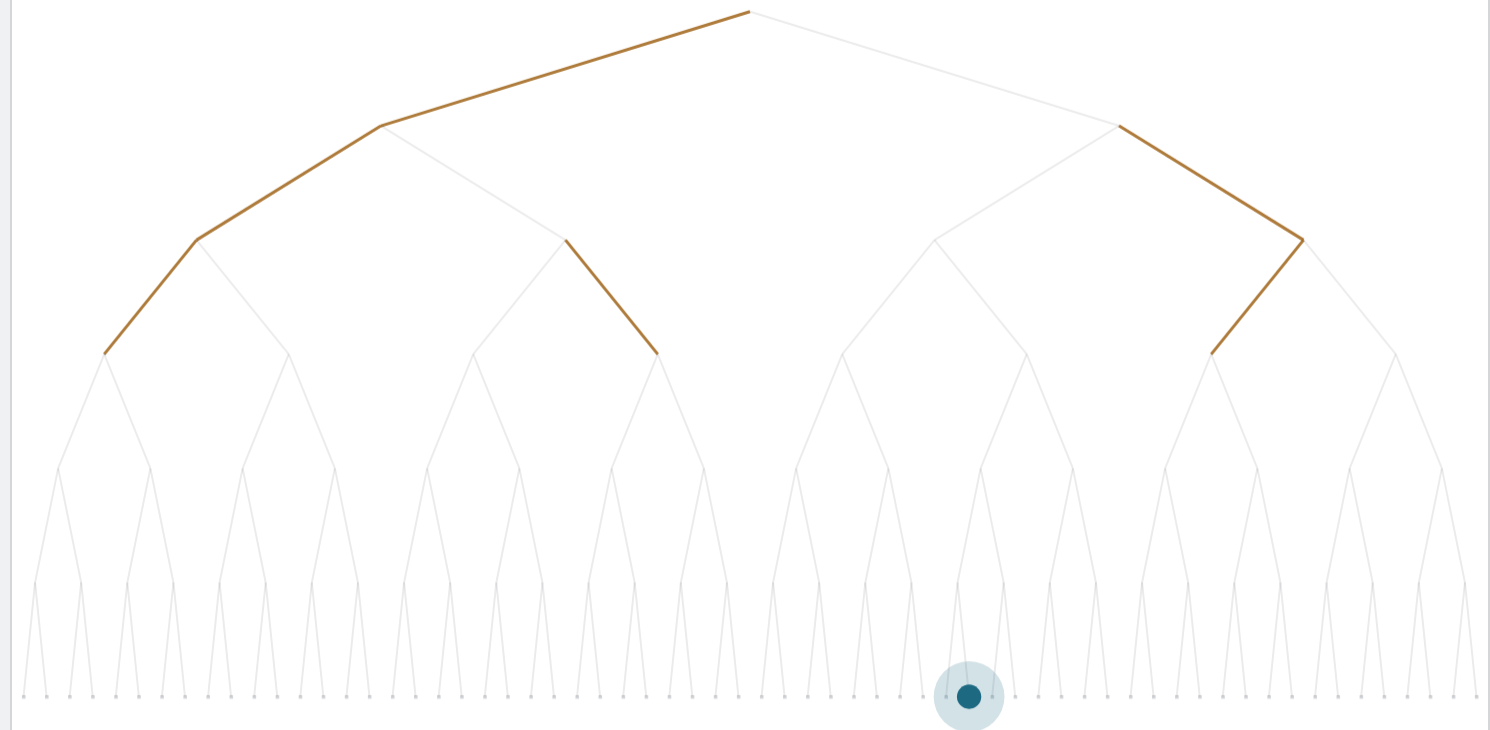
 2^{100}
 $\approx 1.3 \times 10^{30}$ ASSIGNMENTS

 10^{13} years

AT A BILLION CHECKS PER SECOND

WHERE YOU MEET IT IN SELFIE

monster and rotor translate RISC-U machine code — including all of selfie — into exactly such formulae, satisfiable when some input makes the code fail. Syntax becomes semantics, and then hits this wall, before the coffee gets cold.

 $n = 6 \rightarrow 64 \text{ leaves.} \quad n = 100 \rightarrow 10^{30}.$


the one that satisfies

 2^n branches · one satisfying leaf

INTRACTABILITY

Hard to **find**. Easy to **check**.

That asymmetry has a name: NP — problems whose solutions are quick to verify even if finding them seems to need exponential search.

Cook and Levin, 1971–73: thousands of such problems are *the same problem* in disguise. Crack one efficiently and you crack scheduling, routing, folding, packing, proving.

Whether that is possible — $P = NP$? — is the most consequential open question in the exact sciences. Most researchers bet no.

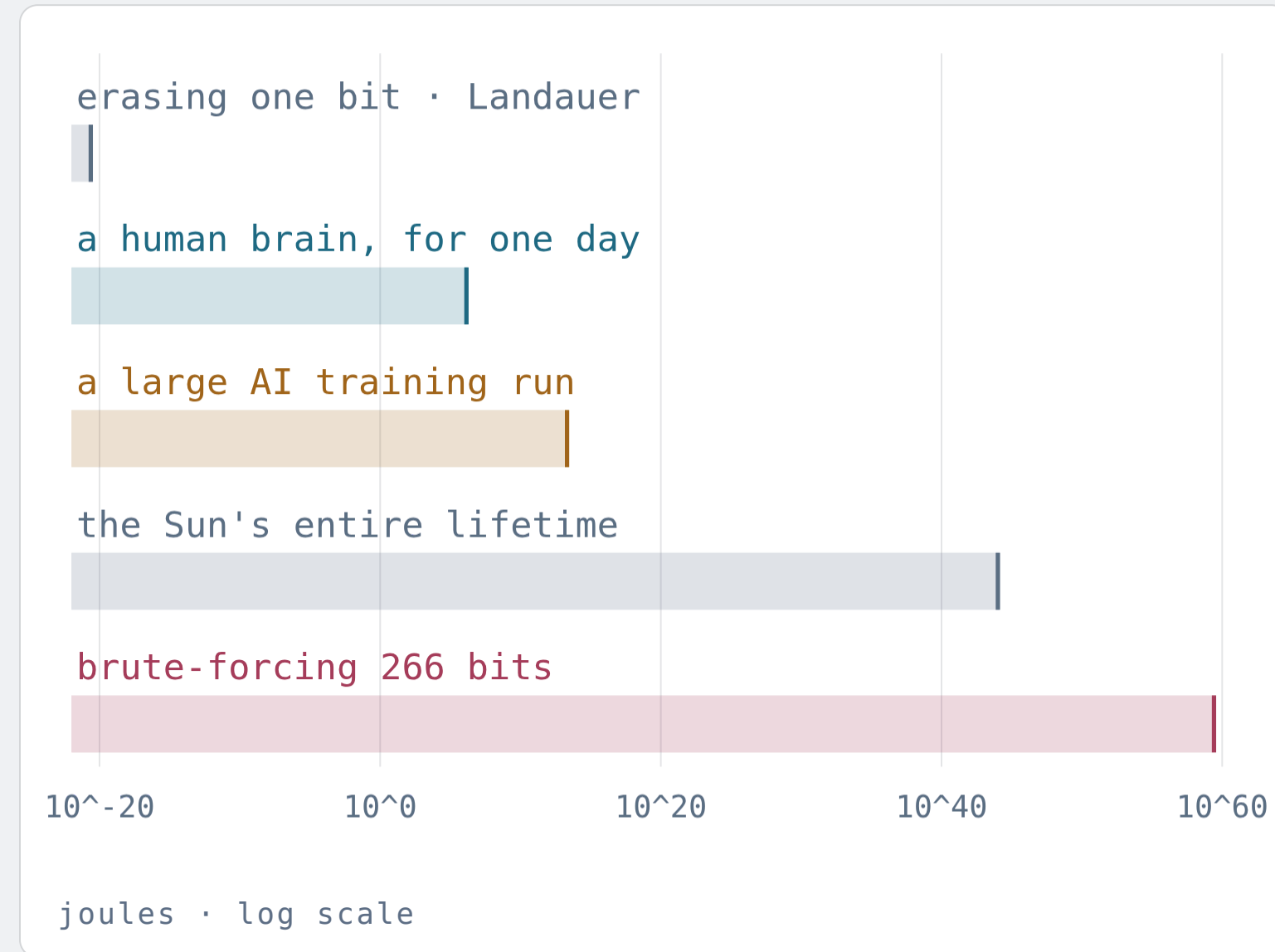
THE ASYMMETRY, EVERYWHERE

Writing a proof versus reading it. Designing a protein versus assaying it. Writing the program versus running the test. Doing the homework versus grading it.

REMEMBER THIS ONE

Generation is expensive; verification is cheap. Every healthy division of labour — and every safe way to use an AI — is built on that gap. And you can only stand on the cheap side if you can *check*.

Computation is not abstract. It costs **energy**, by law.



Landauer, 1961: erasing one bit at room temperature costs at least $kT \ln 2 \approx 3 \times 10^{-21}$ joules. Information is physical.

So brute-forcing our 266-bit space costs $\approx 10^{59}$ J — about 10^{15} times everything the Sun will radiate in its entire lifetime.

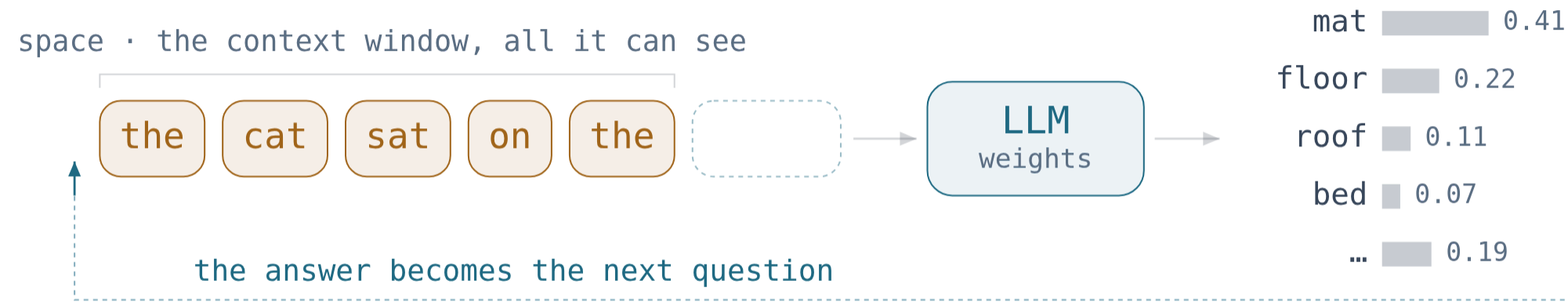
Meanwhile a human brain runs on **20 watts**: a dim light bulb, doing what data centres cannot.

Time, space, energy — three currencies for one budget. Any claim about intelligence that ignores the budget is a claim about magic. Efficient use of a tool means knowing the bill.

Machines

Now the question you came with. Today's AI is remarkable — and it is *subject to every single thing* we have just established.

A large language model is a machine trained on **notation**, asked for **meaning**.



tokens in · a distribution over the next one out · then a sample

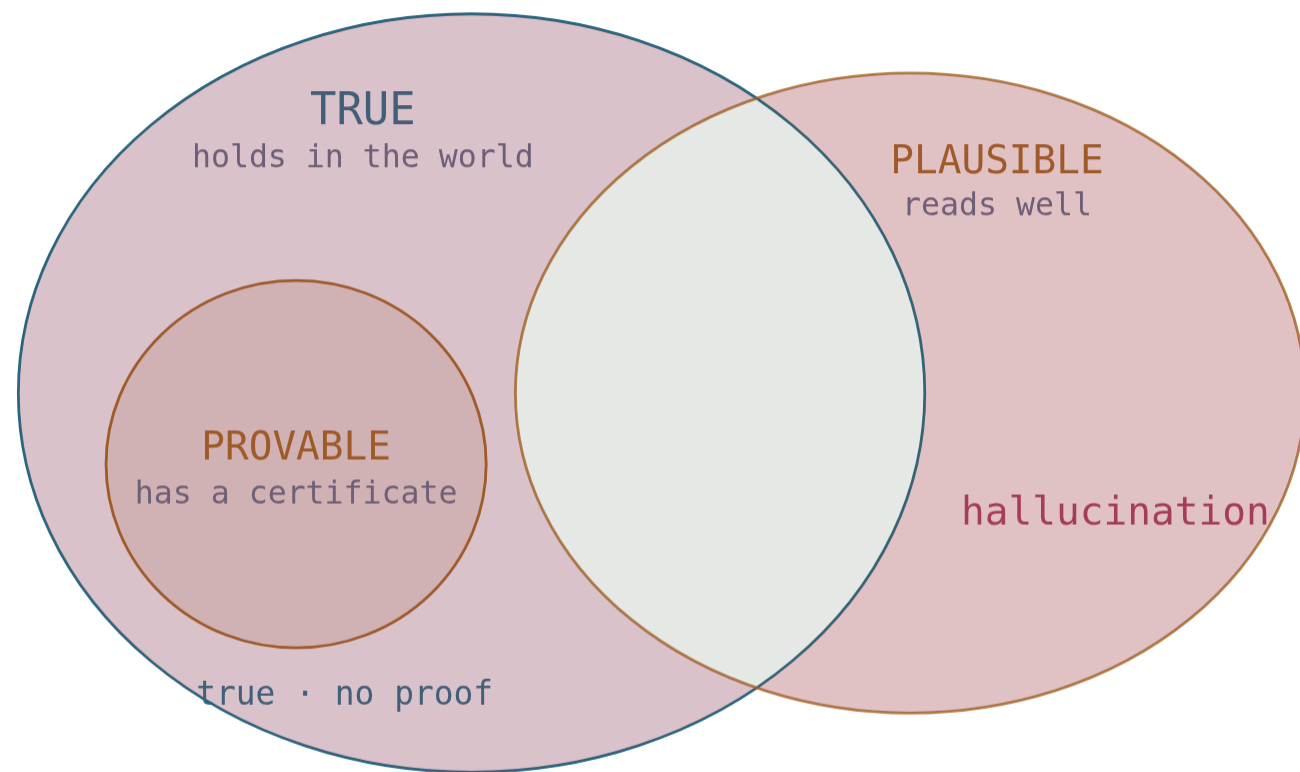
The name is exact. What a large language model — an LLM — is trained on is *language*, so the signal is syntactic: which symbol follows which. Meaning is never handed to it.

That this works as well as it does is the genuine surprise of the decade. It cost megawatts for weeks; you run on twenty watts. Neither figure changes what kind of object it is.

Billions of parameters, each many bits. Part I applies unchanged: that space cannot be inspected, so the behaviour cannot be enumerated. Only sampled.

THE OLD GAP, INDUSTRIALISED

A hallucination is a **proof-shaped object** that isn't **true**.



three tests · and they are not the same test

Fluency is syntax. Correctness is semantics. We built a machine of extraordinary fluency, so the gap between the two is something you now meet before breakfast.

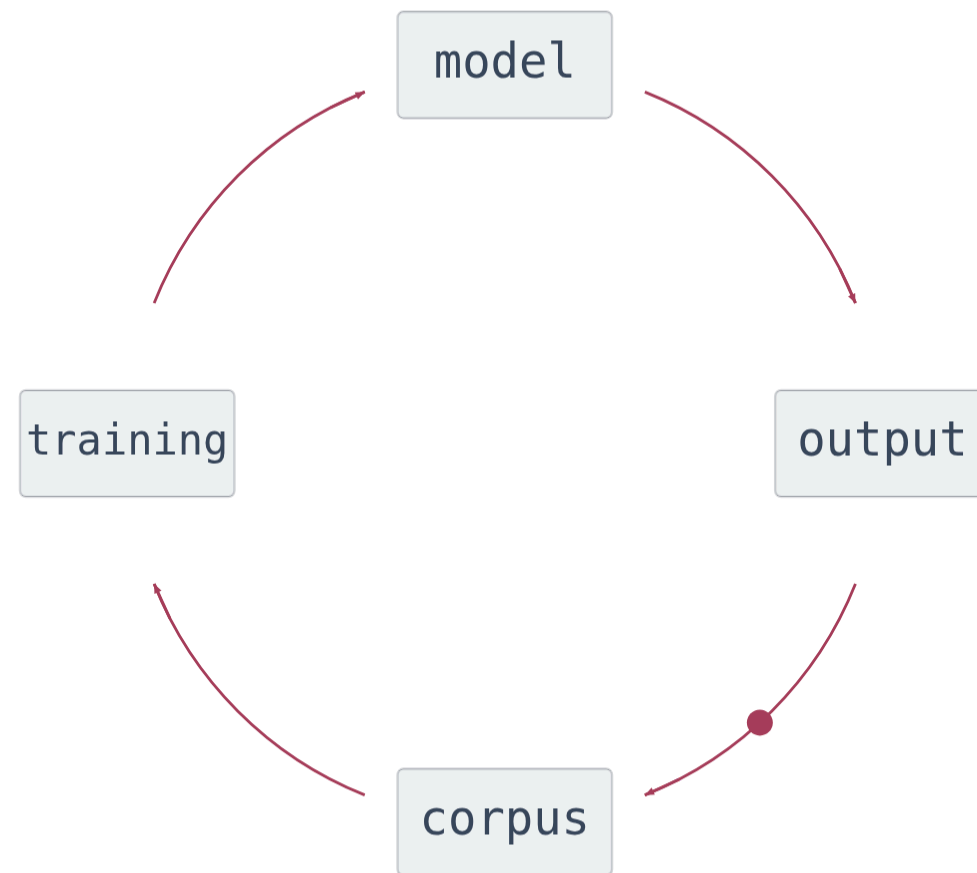
Not a defect to be patched away: it is the proof/truth distinction at consumer scale. Generated code compiles, reads well, and is wrong in the same way a proof-shaped paragraph is wrong.

So the durable response is not "trust it more" or "trust it less" but check it against something with a semantics — a compiler, a test, a proof, a measurement, a colleague.

AND WHO OWNS THE SEMANTICS?

Not the model: it was trained on symbols defined by other symbols. The compiler, the test, the proof — those are yours, if you have learned the domain they belong to.

Generation got cheap. Verification **did not**.



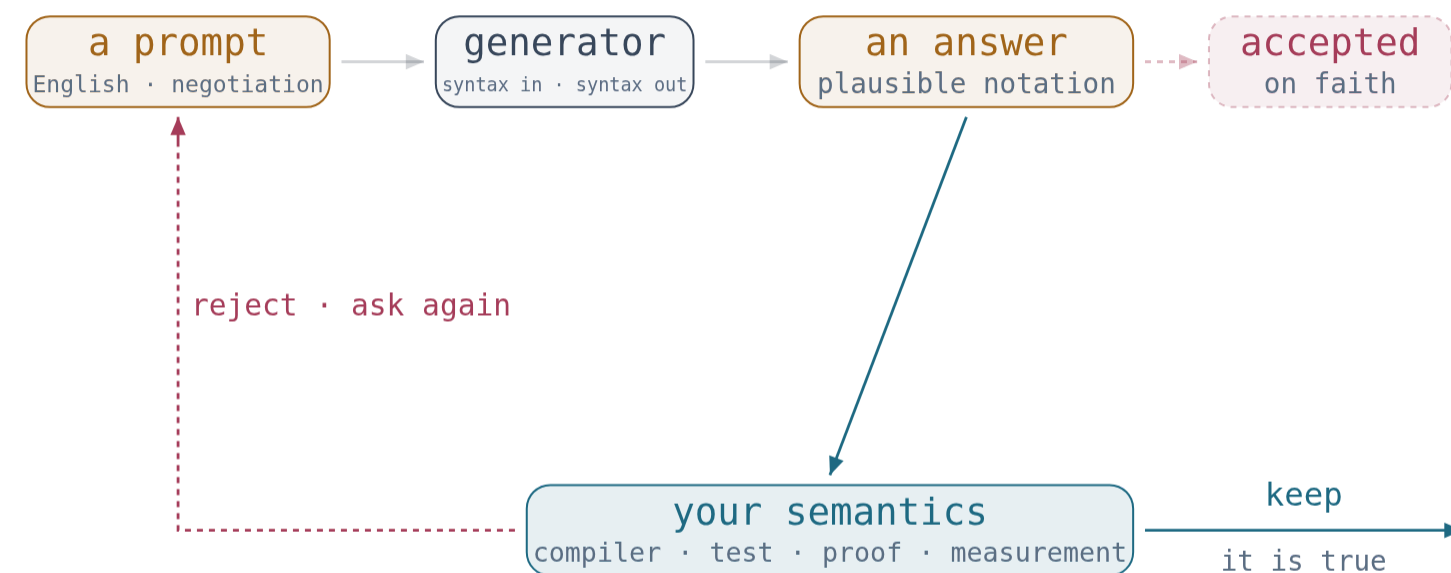
systems judging, training, writing systems

- Producing a program, a proof sketch, a draft was expensive and therefore scarce. Scarcity did our filtering for us. Now production is nearly free and the filter has to be supplied deliberately.
- Models are trained on text that models wrote. Models grade models. Models write the code that trains models. The judge and the candidate share the same blind spots.
- "Is this code correct?" is a semantic property. Rice: no general decision procedure. "Is this system right about itself?" Gödel: no self-certificate. An explanation of its own output is more output.
- Value migrates to the two ends a generator does not occupy: deciding what should be true and establishing that it is. Both are acts of meaning. The better generation gets, the more they are worth.

Why Computer Science

Everything so far was true before this technology arrived and stays true after it. Now the [consequence](#) — and where to start.

A generator is only usable by someone who owns a **semantics** to check it against.



the only part the machine does not supply

what the machine supplies · and what it does not

Prompt in English, and what comes back is **plausible notation**: fluent, confident, and Part V says nothing about whether it is true.

Without a domain, there is only one thing to do with it: **accept it on faith**. Delegation without a check is not delegation.

With one, the answer meets a semantics you own — a compiler, a test, a proof, a measurement — and gets kept or sent back. That gate is the only part of the loop the machine does not supply.

THEREFORE

Effective use — knowing whether the answer is true — needs a semantics (Parts II, III, V). Efficient use — knowing what to ask, in which language, and what it costs — needs the domain's languages and their limits (Part IV). Both live in one place: a nontrivial domain, understood.

WHY THIS ONE

Every field uses languages. Computer science is the field whose **subject** is what languages can and cannot do.

LANGUAGES AND THEIR SEMANTICS ARE THE SUBJECT

A compiler is a semantics you can read and run. A machine is a semantics in silicon. A proof, a type, a test is a semantics you can check. Every other discipline *uses* a notation; this one studies the map from notation to meaning itself.

THE THEOREMS ARE THE USER MANUAL

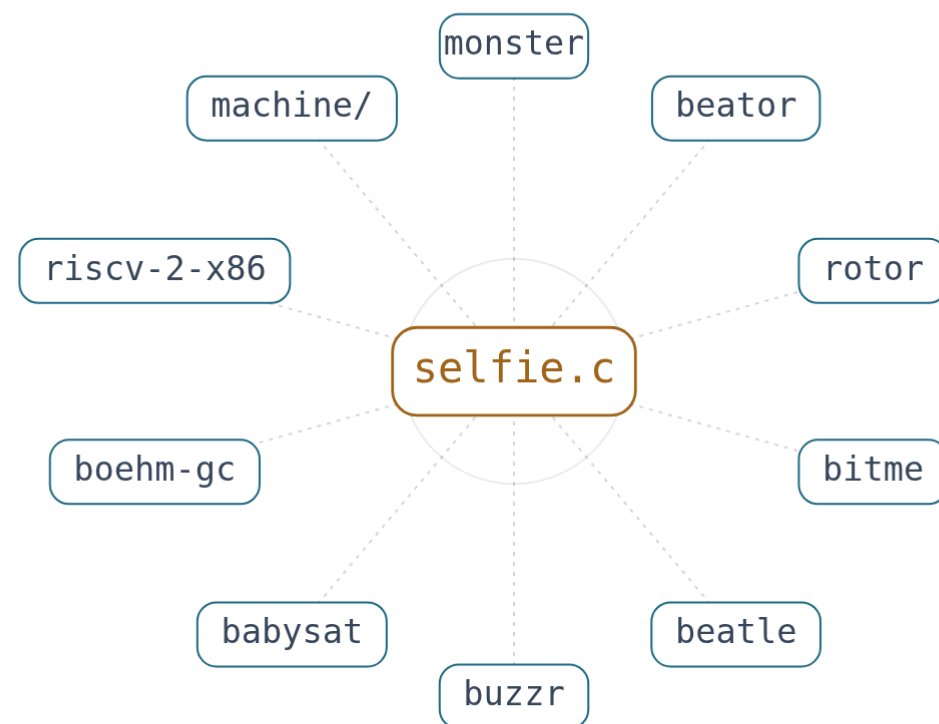
Cantor, Gödel, Turing, Rice, Cook: what any generator can and cannot deliver, proven in advance, for every architecture that will ever exist. You do not have to guess what the next model can do. You can *derive* what no model can.

THE STACK DID NOT DISAPPEAR

Every model that writes your code runs on virtualized machines, compiled by a compiler, scheduled by a kernel, on a processor executing instructions. When it is slow, or wrong, or leaking, the person who can go down there is the person who fixes it.

So the arrival of a machine that generates notation does not make this discipline optional. It makes it **the discipline of using the machine** — which is why modern AI motivates studying it *more* than before, not less.

You do not read the system. You **change** it — and it must still compile itself.



built on selfie · applied to selfie

COMPILER CLASS

Hexadecimal literals. Bitwise operators. `for` loops. Arrays. Structs. Each one is a change to the scanner, the parser and the code generator at once — a new piece of **syntax**, and the **semantics** you give it.

SYSTEMS CLASS

An assembler that assembles itself. Processes, fork, locks, threads, a thread-safe allocator. You are writing the operating system, not reading about one.

ONE RULE ABOVE ALL OF THEM

Whatever you change, **selfie must still compile itself**. Your new code generator has to be good enough to generate the compiler that contains it. A machine may draft your code. This rule decides whether the draft was true — and the public autograder tells you before I do.

The frontier is not **proving**. It is finding **truth** you cannot yet prove.

WORKING DEFINITION • FROM THE COMPANION LECTURE

*Intelligence is developing new formal languages — or at least new properties in existing ones — which requires finding and understanding promising **unproven truth**. New languages and properties let us ask new questions about that truth, and then answer them in **proofs**. Forever.*

MACHINES ARE FORMIDABLE AT

Proving, computing, generating — within a language somebody already built. That is the part that got cheap, and it will keep getting cheaper every year of your career.

NOTHING YET DOES RELIABLY

Finding the **unproven truth** worth formalising, and building the language that captures it. Gödel: no procedure yields the next axiom. Not hard — there is no algorithm there to automate.

Depth is not downloadable. Knowing where a field is thin, where it is wrong, and where it is about to give comes from having lived inside it. Selfie is small enough that you can live inside *all* of it.

CLOSE

Notation is finite. Meaning is not. Someone has to mean it.

The machine writes the notation. You are the part of the loop that knows what it should *mean* and whether it is *true* — and that knowledge is a discipline, learned in depth. This is mine.

OUTCOME A

Confirmed conviction — never to study computer science.

OUTCOME B

Informed confusion. I have seen it turn into a switch *into* the field.

Either way, you now own the argument. Pick the field whose unsolved problems make you *strangely comfortable*. One slide left, and it is a terminal.

TAKE A SELFIE

Fifteen minutes from now you could be running a compiler that compiles itself.

```
$ git clone github.com/cksystemsteaching/selfie  
$ cd selfie && make  
$ ./selfie -c selfie.c -m 2 -c selfie.c
```

No C compiler? `docker run -it cksystemsteaching/selfie`. No terminal at all? It runs in a browser tab.

Then open `selfie.c` and make it print your name. That is assignment one, and it is genuinely how the course starts.

Book — *Elementary Computer Science: From Bits and Bytes to the Universality of Computing*

The long versions — `/intelligence` for the argument in any field · `/talk` for the three commands, explained

Slides, autograder, assignments — all public, all in the repository

`selfie.cs.uni-salzburg.at`
`github.com/cksystemsteaching/selfie`